PROJECT GUIDE · PART 3

StreetCare AI 5-Day Team Build Plan

The dependency-ordered implementation roadmap, checked against the specification and corrected, with a work diagram for every day and each day's work split across a team of five developers.

5 days | 5 developers | 48 modules · 10 packages | 6 entry points | + tests.py | corrections included

---

Companion to *Architecture & Workflow* and *Concepts Handbook*
Prepared for the StreetCare AI team

Contents

1 Is the roadmap complete? Verdict & corrections

2 Why this order: the dependency chain

3 The team: five tracks

4 The week at a glance (who does what)

5 How every day runs

6 Day 1 · Persistence & data cleaning

7 Day 2 · Knowledge base & screening

8 Day 3 · Features, models & tools

9 Day 4 · Agents, workflow & chat

10 Day 5 · Access, API & dashboard

11 Quick reference & final checklist

HOW TO READ THIS

Each day has: a goal, a **work diagram** (one colour per developer, arrows = "needs this first"), a table splitting the work into **five parts**, the exact rules to implement, and a **done-when** checklist ending with the entry point that must run. Developer colours stay the same all week: Dev 1 Dev 2 Dev 3 Dev 4 Dev 5

1. Is the roadmap complete? Verdict & corrections

VERDICT

Yes in structure. The roadmap's order is right, and between its five days it covers **every module and all six entry points**. Its key insight is correct: `run_model_training.py` (entry point 3) needs `read_defect_standard` and `screen_report`, so knowledge and screening must be built on Day 2 before models on Day 3.

Not quite in detail. Checked line by line against `StreetCare_AI.pdf`, `config.py` and the repository, it has a wrong count, three wrong or missing rules, a wrong endpoint count and two missing work items. This plan fixes all of them.

Where	Roadmap says	Correct (from the specification / config)
Summary	39 Python modules across 8 packages	**48 modules across 10 packages**: access 2 · agentic 5 · api 2 · chat 3 · dashboard 7 · data_pipeline 10 · knowledge 4 · modeling 5 · persistence 3 · tools 7. Plus 6 entry points, `tests.py`, and `config.py` (supplied).
Step 2.3	"Verify Section 1 names defect_type and Section 2 matches REQUIREMENTS_SECTION_PATTERN"	The defect type is stated in **§2** (`DEFECT_TYPE_SECTION_NUMBER = 2`); the requirements are in **§4** (`REQUIREMENTS_SECTION_NUMBER = 4`), which must match `REQUIREMENTS_SECTION_PATTERN` whole.
Step 3.1	"median/mean imputation … constant/mode imputation"	Numbers: **median** (`NUMBER_FILL_STRATEGY`) then scaling. Categories: **most_frequent** (`CATEGORY_FILL_STRATEGY`) then one-hot. No mean, no constant.
Step 3.5	six refusal checks starting with missing fields	First: **a screening of another report is refused** (`SCREENING_OF_ANOTHER_REPORT_MESSAGE`); then the first of the six that applies.
Step 5.3	10 endpoints	**11 endpoints**: signin, signout, GET /queue, GET /queue/{ref}, POST /reviews, GET /reviews, POST /reviews/{id}/decision, GET /reviews/{id}/calls, POST /chat/ask, GET /chat/turns, GET /audit/chat-turns.
Missing	tests	`tests.py` is never planned. Here every developer writes tests for their own part daily, and Day 5 adds integration and end-to-end tests.
Missing	secrets	`Project/.env` (real Gemini key) is committed and `build.yml` holds a Sonar token. Rotate both, add `.gitignore`, move the token to repository secrets: **Day 1, first hour**.
Missing	shared test fixtures	A fake Gemini client and temporary DB/checkpoint fixtures are needed by Days 4–5; this plan builds them on Day 4 morning.

Everything else in the roadmap was checked and is consistent with the specification: the 13 repository functions, 14 shared cleaning functions, the expected row counts, chunk overlap with `carried_characters`, the metric names (ROC AUC, Brier score, R^2, MAE vs average estimate, macro/weighted/per-class F1), the 23-key state, automatic function calling off, the 502 failure handler, and the 10-node table.

2. Why this order: the dependency chain

A module is built only after everything it imports exists. Following the arrows gives the five days, and each day ends by running an entry point that proves the day works.

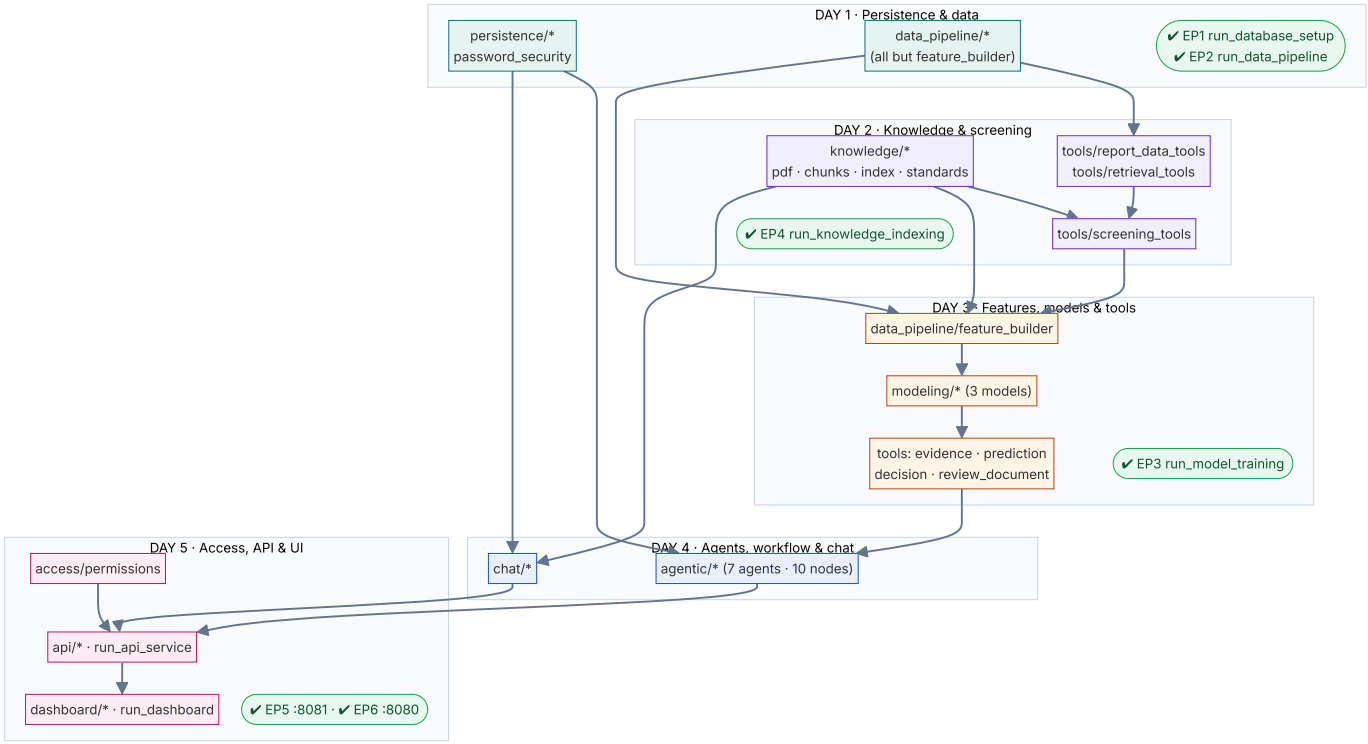


Figure 1: Each day only imports what earlier days built. Note entry point 4 runs on Day 2, before entry point 3 on Day 3.

THE ONE NON-OBVIOUS DEPENDENCY

`run_model_training.py` calls `add_requirement_counts` and `add_coordination_trigger_flags`, which call `screen_report` and `find_coordination_triggers`, which need each type's standard from `read_defect_standard`. So the knowledge base and screening tools (Day 2) must exist before any model is trained (Day 3), even though indexing is entry point 4.

3. The team: five tracks

Each developer keeps one **track** all week, so the same person who builds a layer also integrates it the next day. Work is balanced by function count and difficulty, not by file count.

Dev	Track	Day 1	Day 2	Day 3	Day 4	Day 5
Dev 1	Storage & platform	DB engine + 6 ORM tables	PDF extraction + chunking	feature_builder	state + LLM client + test fakes	permissions + schemas
Dev 2	Records & retrieval	repository + passwords + EP1	vector index + retrieval tools + EP4	registry + evaluation + review documents	5 review agents	routes A (resources, auth, queue) + EP5
Dev 3	Rules & models	loader + cleaning A (structure)	standard reading	repeat + crew hours + EP3	2 writer agents + graph	routes B (reviews, decisions, chat, audit)
Dev 4	Text & live data	cleaning B (readings, text) + 3 cleaners	report data tools + street works + triggers	sentence model + evidence tools	nodes.py (context + 10 nodes)	dashboard client + 3 pages + EP6
Dev 5	Integration & quality	3 cleaners + profiler + EP2 + secrets fix	screen_report	prediction + decision tools	chat (3 modules)	3 pages + end-to-end tests

Why tracks?

Dev 2 builds the repository on Day 1 and the review agents that write through it on Day 4; Dev 4 builds text cleaning on Day 1 and the evidence tools that rely on it on Day 3. Ownership carries forward.

Dev 5 is the integrator

Every day Dev 5 owns the piece that joins the others (the pipeline run, screening, decision rules, chat, end-to-end tests) and runs the day's entry point with the owner.

4. The week at a glance

	Day 1	Day 2	Day 3	Day 4	Day 5
Dev 1	DB engine + 6 ORM tables	PDF extraction + chunking	feature_builder	state + LLM client + fakes	permissions + schemas
Dev 2	repository + passwords ✔ **EP1**	vector index + retrieval tools ✔ **EP4**	registry + evaluation + review documents	5 review agents	routes A + service ✔ **EP5**
Dev 3	loader + cleaning A	standard reading	repeat + crew hours ✔ **EP3**	2 writer agents + graph	routes B (reviews, decisions, chat)
Dev 4	cleaning B + 3 cleaners	live data + street works + triggers	sentence model + evidence	nodes.py (10 nodes)	dashboard A ✔ **EP6**
Dev 5	3 cleaners + profiler + secrets ✔ **EP2**	screen_report	prediction + decision tools	chat (3 modules)	dashboard B + end-to-end tests

Figure 2: Twenty-five work packages, five per day, five tracks (✔ = the developer who runs that entry point)

Day	Modules finished	Entry point verified at end of day	Proof
1	persistence 3 · password_security · data_pipeline 9	EP1 run_database_setup · EP2 run_data_pipeline	16 accounts (then 0 / 16); 5 datasets with exact kept counts
2	knowledge 4 · report_data_tools · retrieval_tools · screening_tools	EP4 run_knowledge_indexing	16 documents, 136 sections; §6 of each standard read back; worked example → INSPECT ON SITE
3	feature_builder · modeling 5 · evidence · prediction · decision · review_document	EP3 run_model_training	3 models above minimums; behavioural checks pass; metrics JSON
4	agentic 5 · chat 3	(no entry point) workflow + chat tests	5 calls before pause; 7/6/5 totals; resume after rebuilding the graph
5	access/permissions · api 2 · dashboard 7 · tests.py	EP5 run_api_service · EP6 run_dashboard	role × route matrix; full review opened and decided from the UI

5. How every day runs

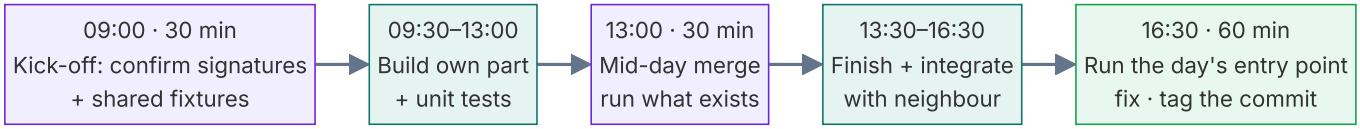


Figure 3: One rhythm, five days

Code against the spec, not each other

Every function signature, parameter, return shape and message is fixed by `StreetCare_AI.pdf` and `config.py`. Two people can build a caller and a callee in parallel; a 5-line stub unblocks anyone waiting.

One branch per part

`day2/dev3-standard-reading`. Merge to `main` only at the mid-day and end-of-day points, after tests pass.

Tests travel with code

Each part ships with its tests in `tests.py` (one section per module). The spec's expected figures are the acceptance tests.

Rules for everyone

Import every constant from config; never type a requirement figure; refuse with the configured message; return new tables; plain values in replies.

STOP THE LINE IF...

an entry point's numbers don't match the specification (row counts, 16 / 136, 16 accounts, 5 / 6 / 7 calls). Fix before moving on: every later day builds on these.

DAY 1

Persistence layer & raw data cleaning

13 modules · Entry points 1 and 2

Goal: `database/streetcare.db` with 6 tables and 16 seed accounts, and the five raw extracts cleaned into `processed/` with `output/data_quality_report.json`.

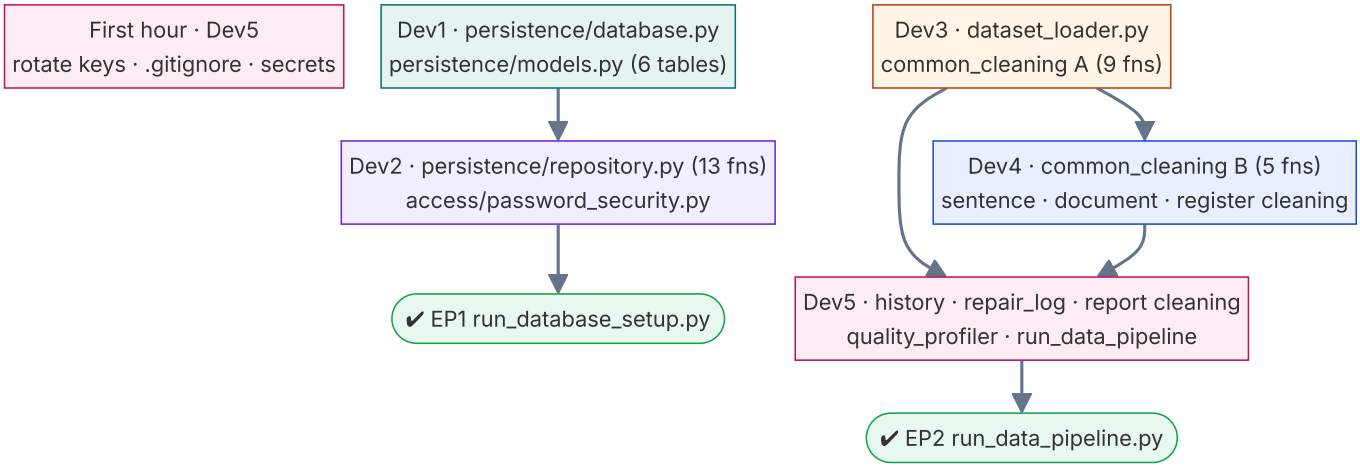


Figure 4: Day 1 work diagram (arrows = needs it first; stubs from the spec unblock parallel work)

Dev	Modules	Functions	Rules to get exactly right
Dev 1	`persistence/database.py` `persistence/models.py`	`database_engine`, `session_factory`; `Base`, `UserAccount`, `AgentRun`, `ReportReview`, `ReviewDecision`, `LlmCall`, `ChatTurn`	Engine at `STREETCARE_DATABASE_URL` with `check_same_thread=False`; no tables created on import. Six tables with exact lower-case names and columns (report_reviews has 37); only nullable columns accept nothing.
Dev 2	`persistence/repository.py` `access/password_security.py` `run_database_setup.py` (EP1)	13 repository functions (insert/fetch user, update token, run insert/update, LLM call insert/fetch, review insert/fetch, decision insert, record outcome, chat insert/fetch); `hash_password`; `main`	Writes join the caller's transaction, never commit on their own; reads return plain dicts; `fetch_review_records` joins run and decision fields. PBKDF2-HMAC-SHA256, 200,000 rounds, 16-byte salt, lower-case hex. Setup refuses a stale schema, is idempotent, prints each account.

Dev 3	`data_pipeline/dataset_loader.py` `common_cleaning.py` part A	`load_raw_dataset`, `save_processed_dataset`, `load_processed_dataset`; `remove_duplicate_records`, `refuse_missing_values`, `refuse_repeated_references`, `remove_rows_missing_target`, `normalize_category_values`, `convert_report_references`, `refuse_unreadable_identifiers`, `convert_whole_numbers`, `keep_latest_revision_of_each_report`	Raw: all text, stripped, `NULL_TOKENS` (any case) → NaN. Processed: only empty cells are NaN so `"None"` stays text. Never mutate inputs; reset index; refuse unknown values naming column + values. References → `SC` + 7 digits. Latest revision wherever rows sit; refuse duplicate (reference, revision).

Dev 4	`common_cleaning.py` part B
	`sentence_cleaning.py`
	`document_cleaning.py`
	`register_cleaning.py`
		`convert_readings`, `treat_values_below_minimum_as_missing`, `treat_values_above_maximum_as_missing`, `convert_markup_to_plain_text`, `lower_text_written_in_capitals`; `clean_sentence_dataset`, `clean_document_records`, `clean_register_records`	Exact `Decimal`, half up once at the reading's step: 4.5 cm → 45 · 1.75 in → 44 · 4 ft → 1.2 · 45 min → 0.8. Strip `MARKUP_PATTERN`, decode entities, collapse whitespace incl. no-break spaces. All-caps → lower. Sentences: recount length, drop untyped and < 3 words. Register: 29 → 28 permits.

Dev 5	`history_cleaning.py`
	`repair_log_cleaning.py`
	`report_cleaning.py`
	`quality_profiler.py`
	`run_data_pipeline.py` (EP2)
		`read_history_answers`, `clean_history_dataset`, `read_log_answers`, `clean_repair_log_dataset`, `clean_report_records`; `profile_dataset`, 3 `profile_*_details`, `write_quality_report`; `DATASET_JOBS`, `main`	History: latest revision first, then drop if it has no `reported_again`. Logs: refuse blank or repeated reference; drop missing `actual_crew_hours`. Live queue 45 → 40. Removal reasons sum to `rows_removed`; JSON indent 2, sorted keys. **First hour:** secrets fix.

Done when

- ☐ EP1 prints 6 tables and *accounts created: 16, already present: 0*; a second run prints *0 / 16*.
- ☐ EP2 prints exactly: history train 32,738 → 30,569 · eval 8,202 → 7,648 · logs train 26,095 → 25,537 · eval 6,476 → 6,337 · sentences 36,661 → 35,717.
- ☐ `data_quality_report.json` written twice gives the identical file; removal reasons add up.
- ☐ Unit tests: every null token; 4 conversion examples; three reference notations; latest revision out of order; duplicate revision refused.
- ☐ Gemini key rotated, `.env` out of git, Sonar token in secrets.

DAY 2

Knowledge base, runtime standards & screening

7 modules · Entry point 4

Goal: the 16 PDFs extracted, split into 136 sections, chunked and embedded into `database/chroma_store/`; each defect standard read at run time; live data loaded; every open report screenable.

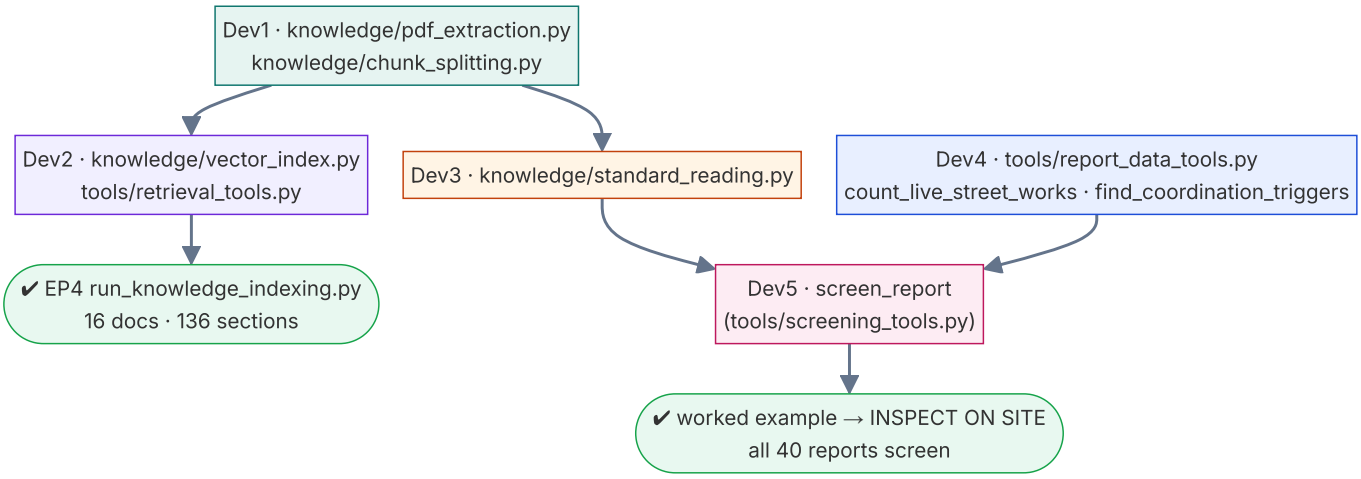


Figure 5: Day 2 work diagram

Dev	Modules	Functions	Rules to get exactly right
Dev 1	`knowledge/pdf_extraction.py` `knowledge/chunk_splitting.py`	`extract_text_from_directory`; `split_documents_into_sections`, `build_chunk_records`	Files ending `PDF_FILE_SUFFIX` (any case), name order, pages joined by `\n`; refuse empty folder / 0-page PDF / blank page. One header line (`DOCUMENT_HEADER_PATTERN`), sections numbered from 1 (`SECTION_HEADING_PATTERN`), 136 total. Passages: whole sentences of one section up to 700 chars; next passage repeats whole trailing sentences ≤ 120 chars; record `chunk_id`, `passage_number` (from 0), `carried_characters`.
Dev 2	`knowledge/vector_index.py` `tools/retrieval_tools.py` `run_knowledge_indexing.py` (EP4)	`build_vector_index`, `open_vector_index`, `read_document_section`, `search_documents`; `read_reviewer_checks`, `read_inspection_records`, `read_works_order_guidance`, `read_resident_update_guidance`, `read_policy_section`; `main`	Persistent Chroma, collection `street_care_documents`, MiniLM on CPU, cosine. Rebuild replaces, never adds. `read_document_section` rebuilds a whole section by stripping `carried_characters`. Search excludes the casebook. Each section must start "N. Title" (else `SECTION_TITLE_MISMATCH_MESSAGE`); return it without the heading. EP4 prints §6 of all five standards.

Dev 3	`knowledge/standard_reading.py`	`read_defect_standard`	Only the configured document. **§2** states the defect type and no other; **§4** matches `REQUIREMENTS_SECTION_PATTERN` whole. Each requirement fits exactly one of `REQUIREMENT_SENTENCE_PATTERNS`; carries kind, name, sentence; rules as Decimals under minimum / maximum / footway_limit / carriageway_limit. Same result every time.
Dev 4	`tools/report_data_tools.py` `screening_tools.py` part A	`load_open_reports`, `load_report_documents`, `load_street_works_register`, `load_report_history`, `fetch_report`, `fetch_report_documents`, `rank_report_queue`, `describe_record`; `count_live_street_works`, `find_coordination_triggers`	40 open reports, 28 permits (23 live). Queue: Footway (type or linked) first, then the rest, each by reference; never by a model figure. `describe_record`: missing → `NOT_GIVEN_TEXT`. Only live permits on the same street; conflicting = same highway part; vegetation in neither. Blank street category = traffic-sensitive; blank column height = taller than 8 m.
Dev 5	`screening_tools.py` part B	`screen_report`	Own standard first, then linked; each requirement keeps its standard. Met / Not met / Not documented by kind (yes/no, at least, at most, less than, water reach, clear height, column lean) with exact Decimals; Not documented names only the blanks it needs. Assessment: any Not met/Not documented → INSPECT; else trigger → REFER; else ORDER. `decisions_open` from config.

Done when

- ☐ EP4 prints *16 documents, 136 sections, N passages* and five different §6 texts, one per standard.
- ☐ Carriageway and Footway standards read different depth figures; a standard of the wrong type is refused.
- ☐ Report Rules §13 worked example screens to Met 1 · Not met 2 · Not documented 1 → INSPECT ON SITE.
- ☐ All 40 live reports screen without error; the queue order puts Footway-linked reports first.

DAY 3

Feature builder, three models & remaining tools

10 modules · Entry point 3

Goal: three gated, saved pipelines with schemas and `output/model_metrics.json`, plus every deterministic tool the agents will use.

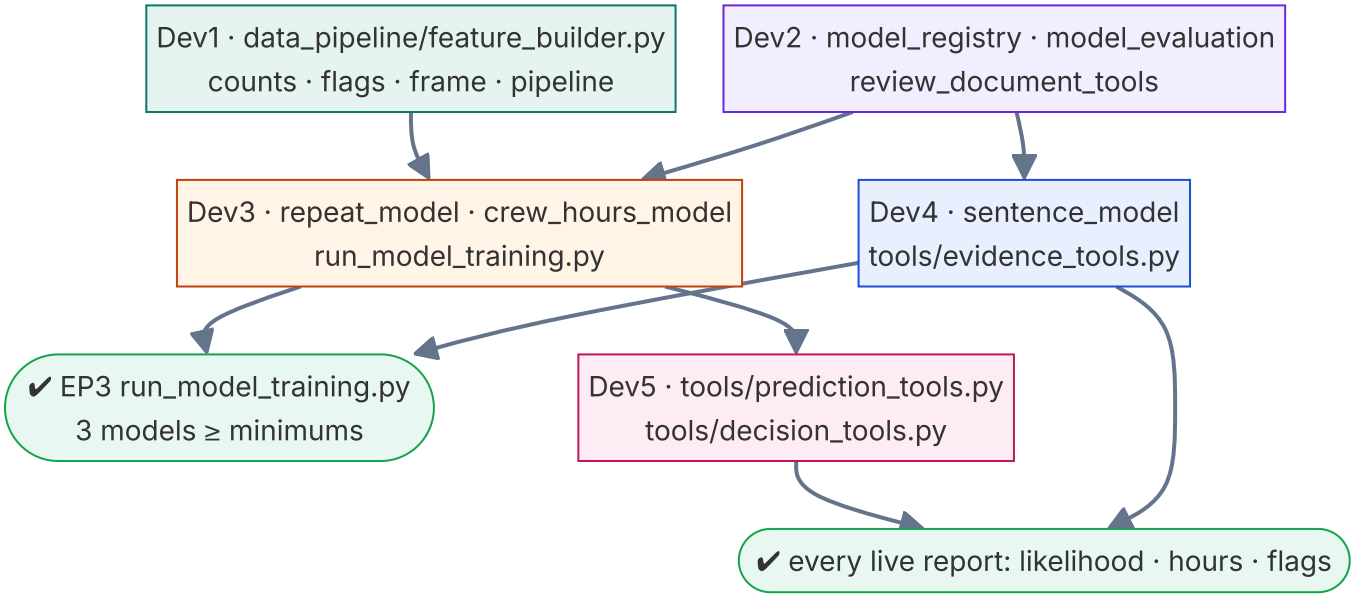


Figure 6: Day 3 work diagram

Dev	Modules	Functions	Rules to get exactly right
Dev 1	`data_pipeline/feature_builder.py`	`add_requirement_counts`, `add_coordination_trigger_flags`, `build_feature_frame`, `build_model_pipeline`	Three counts via `screen_report`; three 0/1 flags via `find_coordination_triggers`; nothing else changes. Frame: numeric then categorical in the given order, floats with NaN, categories with NaN (never None/"nan"/""). Pipeline steps `preprocessing` → `estimator`: **median** + scale; **most_frequent** + one-hot (ignore unknown).
Dev 2	`modeling/model_registry.py` `modeling/model_evaluation.py` `tools/review_document_tools.py`	`save_model`, `save_feature_schema`, `load_model`, `load_feature_schema`, `save_metrics`; `evaluate_classifier`, `evaluate_regressor`, `evaluate_text_classifier`; `build_works_order_details`, `build_inspection_request_details`, `build_coordination_brief`	joblib compress 3; schemas JSON indent 2; metrics sorted keys. Never refit. Classifier: train/held-out ROC AUC, gap, Brier, mean predicted probability, positive share. Regressor: R^2, gap, MAE, average-estimate MAE, lowest/highest. Text: macro F1, gap, accuracy, weighted F1, per-class F1. All rounded to `METRIC_DECIMALS`. Document details from config column lists.

Dev 3	`modeling/repeat_model.py` `modeling/crew_hours_model.py` `run_model_training.py` (EP3)	`train_repeat_model`, `train_crew_hours_model`, `main`	LogisticRegression (3,000 iter); RandomForest (300 trees, leaf 10, max features 0.7); seed 1313; fit on train only. ROC AUC ≥ 0.80 (held-out mean within 0.03 of training); R^2 ≥ 0.86 and MAE < 0.75 × baseline; no forecast below the fewest logged hours. Behavioural checks (≥ 95% / ≥ 90%). Any below minimum → stop, save nothing.
Dev 4	`modeling/sentence_model.py` `tools/evidence_tools.py`	`train_sentence_model`; `read_document_evidence`, `link_evidence_to_requirements`, `find_disagreements`	TF-IDF → LogisticRegression on raw text, 20% held out, nine classes, every class predicted; macro F1 ≥ 0.95. Split documents at `STATEMENT_SENTENCE_PATTERN`; link via `SENTENCE_TYPE_REQUIREMENT_KINDS`; only Measurement sentences searched with the four patterns; unrounded Decimal compare → `DESCRIBES BLANK ANSWER` / `DISAGREES WITH FORM`; never change an answer.
Dev 5	`tools/prediction_tools.py` `tools/decision_tools.py`	`estimate_repeat_likelihood`, `estimate_expected_crew_hours`, `describe_expected_crew_hours`; `find_review_conflict`, `find_decision_refusal`	Lay out in schema order; likelihood half up to 0.01, bands 0.40 / 0.20; hours to 0.1; overrun > 1.3 × planned; three templates. Conflict: already decided / different decision pending. Refusal: **screening of another report first**, then missing fields → unknown decision → not an inspector → not appointed for the street's district → opened the review → decision not open.

Done when

- ☐ EP3 prints held-out, train and gap for all three, and *every model reaches the minimum the configuration sets for it.*
- ☐ Lowering a model's minimum deliberately above its score stops the run with nothing saved (test it once).
- ☐ Across the 40 live reports, likelihoods fall in all three bands and overruns are expected for some, not all.
- ☐ Every refusal message is the configuration's own words; the refusal order is tested one case at a time.

DAY 4

Ten-node LangGraph workflow & street care chat

8 modules · no entry point; proven by workflow tests

Goal: seven single-call Gemini agents, the checkpointed ten-node workflow that pauses at the gate and resumes after a restart, and the guarded chat.

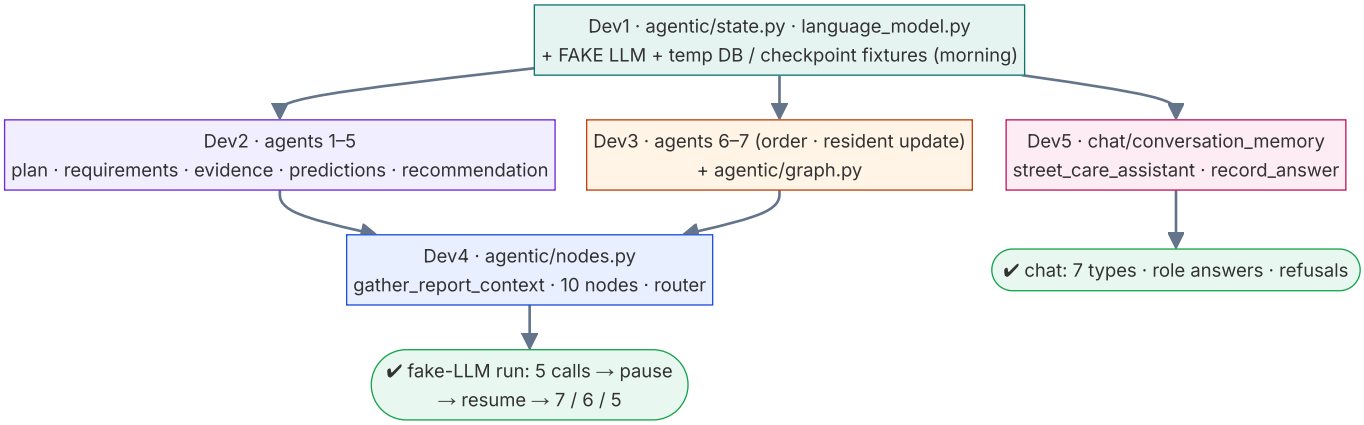


Figure 7: Day 4 work diagram. The fake LLM is the first thing built: everyone tests against it

Dev	Modules	Functions	Rules to get exactly right
Dev 1	`agentic/state.py` `agentic/language_model.py` test fixtures	`ReportReviewState`; `generation_settings`, `call_language_model`, `parse_labelled_response`, `join_labelled_response`; fake Gemini	23 keys and nothing else; `llm_call_count` additive. Max tokens 8192, temperature 0.2, automatic function calling off. Key and model read at call time. Refuse empty, unfinished or token-less replies. Labels read ignoring case and edge decoration; refuse missing/extra/multi-line single-line fields. Latency in whole ms.
Dev 2	`agentic/agents.py` part A	`plan_review`, `analyse_requirements`, `review_evidence`, `analyse_predictions`, `write_review_recommendation`	Speak for the Street Care Office of Wrenford Borough Council; use only given facts (via `describe_record`, `FORM_ANSWER_LABELS`). Exact field names per agent. RECOMMENDED DECISION single-line, upper case, in `decisions_open`. Each returns its fields + `call_record`.
Dev 3	`agentic/agents.py` part B `agentic/graph.py`	`draft_order_document`, `draft_resident_update`; `build_workflow`	SUBJECT/DOCUMENT, HEADLINE/UPDATE; name required references/street/defects; no forbidden terms, no `[ ]`, no staff names; end with the sign-off. Graph: ten names exact, first seven in order, two conditional branches via `route_report_decision`, END after node 10, SQLite checkpointer with a multi-thread connection; building runs nothing.

Dev 4	`agentic/nodes.py`	module resources, `gather_report_context`, 10 node functions, `route_report_decision`	Load models, live files and index once on import. Node 1: RUN-/REV- + 12 hex, run RUNNING, no LLM. Each agent node records its call immediately, returns `llm_call_count = 1`. Node 6: call + review AWAITING + run in one transaction. Node 7: `interrupt`, writes nothing, runs `find_decision_refusal` on resume. Node 10: decision + outcome + run COMPLETE in one transaction.

Dev 5	`chat/conversation_memory.py`
	`chat/street_care_assistant.py`
	`chat/record_answer.py`
		`recall_conversation`, `answer_from_documents`, `answer_report_record`, `answer_street_figure`	Last 3 own turns, record answers masked. Two searches together, top 4, casebook excluded; one call classifies into 7 types; answer only if distance ≤ 0.64, else refusal. Report questions answered by code by role (advisers: no predictions, `OUTCOME_NOT_PREDICTED_LINE`). Street figure from history, by defect type, only if ≥ 5.

Done when

- ☐ With the fake LLM: a run makes 5 calls, pauses at node 7, and the review is AWAITING_DECISION in the DB.
- ☐ Rebuilding the graph from `checkpoint.db` (new process) and resuming finishes with 7 / 6 / 5 calls for ORDER / INSPECT / REFER.
- ☐ Each refusal at the gate returns the configured message and spends no call.
- ☐ Chat: adviser asking a prediction gets the fixed line; a street with < 5 reports gets `GROUP_SIZE_LINE`; out-of-scope is refused.
- ☐ One live Gemini smoke test passes (key from `.env`).

DAY 5

Access control, REST service & dashboard

10 modules + tests.py · Entry points 5 and 6

Goal: every route guarded by role and district, the API on `:8081`, the dashboard on `:8080`, and a full review opened and decided end to end.

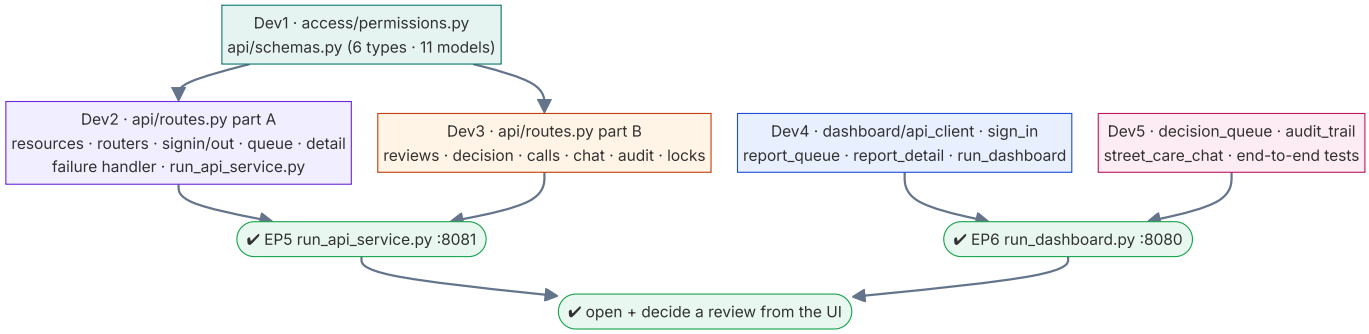


Figure 8: Day 5 work diagram. The dashboard is built against the API's documented replies in parallel

Dev	Modules	Functions	Rules to get exactly right
Dev 1	`access/permissions.py` `api/schemas.py`	`authorize_request`, `authorize_district`; `DefectTypeFilter`, `ReviewStatusValue`, `DecisionValue`, `CoordinatorRequestText`, `DecisionReasonText`, `QuestionText` + 11 models	`X-Session-Token`; missing/unknown → **401** `SIGN_IN_REQUIRED_MESSAGE`; permission from `PERMISSION_BY_ROUTE` not in role → **403**. Inspector outside their district → 403 before any run or call. Exact field names; question 3–1,000 chars; plain values only.
Dev 2	`api/routes.py` part A `run_api_service.py` (EP5)	`review_workflow`, `report_history`, `report_profiles`, `report_details`, `report_queue`, `report_locks`, `public_router`, `guarded_router`, `service_application`; `sign_in_user`, `sign_out_user`, `list_report_queue`, `get_report_detail`, `report_failed_request`; `main`	Queue and details computed once on import (0 LLM calls). Guarded router includes `authorize_request`. 200 everywhere except POST /reviews (201). Unknown report → 404. Failure handler → **502** with the failure's message and the opened run marked FAILED. EP5 checks `SERVICE_REQUIRED_FILES` first and names what's missing.
Dev 3	`api/routes.py` part B	`start_report_review`, `list_report_reviews`, `submit_review_decision`, `get_review_calls`, `ask_street_care_assistant`, `list_own_chat_turns`, `list_recent_chat_turns`	Per-report `threading.Lock`. Decision: under the lock, `find_review_conflict` → 409; refusal → 409; else resume from checkpoint; the same decision after a failure finishes the run. Calls reply: count vs expected (5/6/7), total tokens, total latency. Only the three POSTs may spend LLM calls.
Dev 4	`dashboard/api_client.py` `sign_in.py`, `report_queue.py`, `report_detail.py` `run_dashboard.py` (EP6)	`call_service`, `render_sign_in_page`, `render_report_queue_page`, `render_report_detail_page`, `PAGE_RENDERERS`, `main`	Every request through `call_service`: token attached, 300 s timeout, spinner with elapsed time on LLM calls, 401 → back to sign-in, 502 shown with the failure message. Pages only from `ROLE_PAGE_ACCESS`; icons and URL paths from config; no business rules.

Dev 5	`decision_queue.py`, `audit_trail.py`, `street_care_chat.py` `tests.py` integration + e2e	`render_decision_queue_page`, `render_audit_trail_page`, `render_street_care_chat_page`; test suites	Decision page hides reviews the user opened or outside their district (the API still enforces). Audit: reviews, calls with tokens and latency, chat turns (max 200). Chat page: last 20 turns. Tests: role × route matrix, district and own-review refusals, full open → decide → documents run.

Done when

- ☐ EP5 prints the docs URL; with a model file removed it prints *The service cannot start* and names the file and its step.
- ☐ Role × route matrix passes: adviser 403 on queue; coordinator 403 on decision; no token 401.
- ☐ From the UI: coordinator opens a review (201, 5 calls); a second inspector of that district orders the repair (7 calls, REPAIR_ORDERED, works order + resident update stored).
- ☐ The opener cannot decide; an inspector of another district gets 403; a second decision gets 409.
- ☐ Restart the API while a review waits; deciding afterwards still completes it.

11. Quick reference & final checklist

The ten nodes (Day 4)

#	Registered name	Agent / calls	Returns · writes
1	`load_report_context`	none · 0	run_id, review_id + 8 context keys · agent_runs RUNNING
2	`plan_report_review`	plan_review · +1	review_plan (PRIORITY, CHECKS, RATIONALE) · llm_calls
3	`assess_report_requirements`	analyse_requirements · +1	requirements_analysis (5 fields) · Policy §6 · llm_calls
4	`analyse_report_statements`	review_evidence · +1	evidence_review (4 fields) · guides §3 · llm_calls
5	`interpret_model_predictions`	analyse_predictions · +1	prediction_analysis (3 fields) · llm_calls
6	`prepare_report_recommendation`	write_review_recommendation · +1	one transaction: call + review AWAITING_DECISION + run (5 calls) · Policy §7
7	`await_inspector_decision`	human gate · 0	interrupt; on resume find_decision_refusal → review_decision
8	`draft_order_document`	draft_order_document · +1	ORDER or INSPECT: order_document, document_details
9	`draft_resident_update`	draft_resident_update · +1	ORDER only: resident_update
10	`record_report_outcome`	none · 0	one transaction: decision, status, documents (brief if REFER), run COMPLETE (5/6/7)

Role access (Day 5)

Permission / route	Highways Inspector	Street Care Coordinator	Customer Services Adviser
queue · GET /queue, /queue/{ref}	✔	✔	403
review · POST /reviews	✔ (own district)	✔	403
decision · POST /reviews/{id}/decision	✔ own district, not own review	403	403
audit · GET /reviews, /reviews/{id}/calls, /audit/chat-turns	✔	✔	403
chat · POST /chat/ask, GET /chat/turns	✔ full records	✔ full records	✔ status only, no predictions

Friday sign-off

- ☐ All six entry points run in order from a clean checkout (delete `processed/`, `output/`, `database/` first).
- ☐ `python3 -m pytest tests.py` passes with the fake LLM; one live Gemini test passes.
- ☐ Every function and module name matches the specification exactly (the marking suite imports them).
- ☐ No requirement figure anywhere in code or config; no secret in git.